Informe de seguridad de la aplicación móvil MackeUp AI

Auditoría con el OWASP Mobile Top 10 (2024)

Materia:	Programación Móvil
Profesor:	Alex Vacca
Integrantes:	Karlos Rodolfo Quintero Vargas Jaider Enrique Monrroy Diaz Ana Laura Cuellar Hinojosa Oscar Mauricio Parra Cañabate
Modalidad:	Trabajo en grupo, sobre el proyecto de la materia
Repositorio:	github.com/Karquin06/ProjectMake-UpAi — rama `feature/seguridad`
Entrega:	**16 de noviembre de 2026**

Reglas: se probó solo la app y el backend propios. Las evidencias de este informe ocultan llaves, tokens, contraseñas y datos personales; los reportes completos de las herramientas están en `auditoria/evidencia/`, que no se sube al repositorio.

Resumen

Este informe presenta la auditoría de seguridad de MackeUp AI, una aplicación Flutter de asesoría de imagen con backend en Firebase, realizada con la lista OWASP Mobile Top 10 (2024). Se elaboró un modelo de amenazas, se revisaron los diez riesgos con siete técnicas (búsqueda de secretos con gitleaks, análisis de dependencias, revisión del manifiesto, análisis del APK de release, análisis estático con MobSF, revisión de código de las reglas y de las Cloud Functions, y Logcat en un emulador) y se registraron nueve hallazgos: uno de severidad alta, cinco medios y tres bajos. El más grave fue que el APK de release se firmaba con una llave publicada en el repositorio público. Se corrigieron ocho hallazgos, cada uno en su propio commit, y se verificaron repitiendo la misma prueba antes y después de la corrección. Siete quedaron cerrados por completo, uno quedó cerrado de forma parcial con un riesgo residual justificado, y la integración con App Check quedó como mejora pendiente. Además se implementaron dos mejoras preventivas: certificate pinning con plan de rotación y protección de las pantallas que muestran el rostro con `FLAG_SECURE`.

Palabras clave: seguridad móvil, OWASP Mobile Top 10, Flutter, Firebase, auditoría

Introducción

Las aplicaciones móviles manejan cada vez más datos personales y se ejecutan en dispositivos que su creador no controla: cualquier persona puede descargar el APK, inspeccionarlo, modificarlo o interceptar su tráfico. Por eso la seguridad de una app no puede depender de lo que la interfaz muestra u oculta, sino de controles verificables en el código, en la configuración del build y, sobre todo, en el servidor. El OWASP Mobile Top 10 reúne los diez riesgos más frecuentes en este tipo de aplicaciones y es la referencia que se usa en este trabajo (OWASP Foundation, 2024a).

MackeUp AI analiza la colorimetría de la usuaria a partir de una selfie, recomienda maquillaje y ropa, ofrece un simulador de maquillaje en realidad aumentada, un asistente con inteligencia artificial y un módulo para marcas. Su backend usa Firebase Authentication, Cloud Firestore, Cloud Storage, Cloud Functions y Cloud Messaging. La auditoría siguió la guía de la asignatura (Vacca, 2026), que pide modelar las amenazas, auditar la app con el OWASP Mobile Top 10, corregir al menos cinco hallazgos y demostrar el antes y el después.

La regla base del trabajo es que el dispositivo y la aplicación instalada no son de confianza: toda decisión de permisos se valida en el servidor. Como marco de referencia se usaron el OWASP Mobile Top 10 (OWASP Foundation, 2024a), el estándar de verificación MASVS (OWASP Foundation, 2024b) y la guía de pruebas MASTG en su versión 2.0.0 (OWASP Foundation, 2026). Los cambios de la edición 2024 frente a la de 2016 se tomaron del resumen de Holloway-George (2024).

Se auditó la rama `feature/seguridad` del repositorio del proyecto en el commit `28fc2f7` (12 de noviembre de 2026). Las evidencias de este informe ocultan llaves, tokens y datos personales; los reportes completos de las herramientas se guardaron en `auditoria/evidencia/`, carpeta excluida del repositorio.

Objetivos

Objetivo general

Evaluar y mejorar la seguridad de la aplicación MackeUp AI aplicando el OWASP Mobile Top 10 (2024), corrigiendo los hallazgos de mayor severidad y demostrando con pruebas repetibles que las correcciones funcionan.

Objetivos específicos

1. Elaborar un modelo de amenazas de la aplicación que identifique sus componentes, los datos sensibles que maneja y al menos tres amenazas reales.
2. Revisar los diez riesgos del OWASP Mobile Top 10 con al menos dos técnicas de auditoría y registrar cada hallazgo con su evidencia y severidad.
3. Corregir como mínimo cinco hallazgos, empezando por los de mayor severidad y cubriendo las mejoras obligatorias de la guía de la asignatura.
4. Verificar cada corrección repitiendo la misma prueba antes y después, y dejar cada cambio trazable en un commit.

Alcance

La auditoría cubrió la aplicación Android compilada en modo release, el código Dart, las reglas de seguridad de Cloud Firestore y Cloud Storage, y las Cloud Functions del proyecto. No se incluyeron la versión de iOS, que no se distribuye, ni la configuración de la consola de Firebase y Google Cloud, a la que solo se accede con la cuenta de administración. Por la regla de la guía de probar solo la app y el backend propios sin datos reales (Vacca, 2026), no se inició sesión ni se crearon cuentas en el proyecto de producción; las pruebas que lo necesitaban se hicieron en emuladores locales.

Metodología

La auditoría siguió las cuatro fases que propone la guía de la asignatura (Vacca, 2026). En la primera, de modelado, se identificaron los componentes de la aplicación, los datos sensibles y las amenazas reales. En la segunda, de auditoría, se revisaron los diez riesgos del OWASP Mobile Top 10 con varias técnicas y cada riesgo se marcó como *cumple*, *no cumple* o *no aplica*, con su justificación. En la tercera, de corrección, se resolvieron los hallazgos empezando por los de mayor severidad, cada uno en un commit propio. En la cuarta, de verificación, se repitió la misma prueba de cada hallazgo para comprobar que la corrección lo eliminaba.

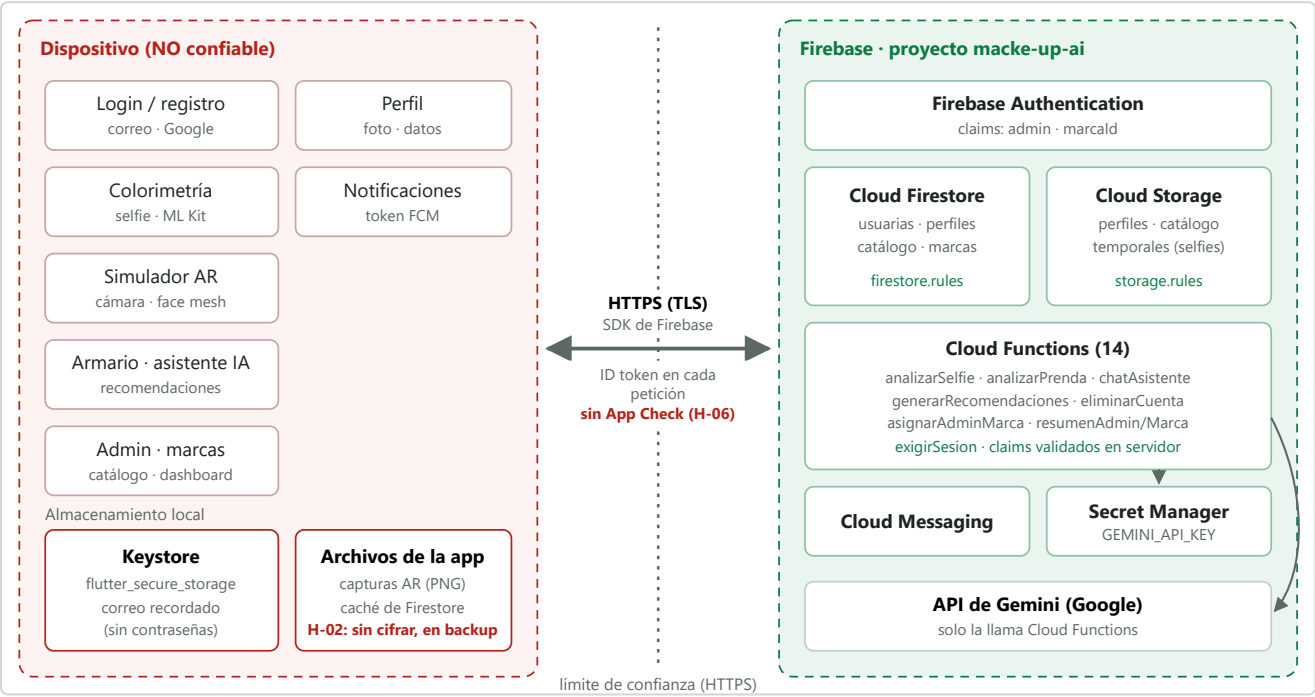
El trabajo fue de tipo aplicado y se apoyó en pruebas reproducibles: cada técnica se automatizó con un script que guarda su salida con la fecha y el commit evaluado, de modo que el resultado de antes y el de después se pueden comparar sobre la misma prueba. Las secciones siguientes describen el modelo de amenazas y las técnicas usadas.

Modelo de amenazas

La Figura 1 muestra los componentes de la aplicación, el límite de confianza entre el dispositivo y Firebase, y dónde se guarda cada dato. La Tabla 1 resume los datos sensibles y la Tabla 2 las amenazas identificadas.

Figura 1

Componentes de MackeUp AI y flujo de datos entre el dispositivo y Firebase



Nota. La única vía entre el dispositivo y el backend es el SDK de Firebase sobre HTTPS, con el ID token de la usuaria en cada petición. Elaboración propia.

Tabla 1

Datos sensibles que maneja la aplicación

Dato	Dónde se guarda	En tránsito	Sensibilidad
Correo y contraseña	Firebase Auth; en el dispositivo solo el correo (Keystore)	HTTPS a Auth	Alta
Sesión (ID token y refresh token)	SDK de Firebase Auth	Cabeceras a Functions	Alta
Selfie y fotos del rostro	Temporal de análisis; capturas AR en archivos de la app	HTTPS a Storage y Functions	Alta (biométrico)
Rol de administradora o de admin de marca	Custom claims del token	—	Alta
Perfil de colorimetría	Firestore `perfiles_colorimetria`	HTTPS	Media
Foto de perfil, conversaciones con la IA, armario	Storage y Firestore	HTTPS	Media
Token de notificaciones (FCM)	Firestore `usuarias/{uid}`	HTTPS	Baja

Tabla 2

Amenazas identificadas y control esperado

ID	Amenaza	Quién	OWASP	Control esperado
A-01	Distribuir un APK modificado firmado con la misma llave que la app real, porque la llave está en el repositorio público.	Cualquiera que clone el repositorio	M1, M8	Llave de release privada (H-01)
A-02	Una marca desactivada o una ex-admin de marca sigue modificando el catálogo.	Ex-administradora de una marca	M3	Reglas que revisen marca activa y lista de admins (H-03)
A-03	Recuperar las fotos del rostro del simulador AR desde el respaldo de Google o una copia del teléfono.	Persona con acceso a la cuenta o al dispositivo	M9, M6	Excluir los datos del respaldo (H-02)
A-04	Llamar a las Cloud Functions con un script, sin la app, para gastar la cuota de Gemini o falsear estadísticas.	Atacante con cuentas creadas en masa	M3	App Check con Play Integrity (H-06)

Nota. Modificar la app para entrar al panel de administración ya está mitigado: las reglas y las funciones validan el claim `admin` en el servidor.

Técnicas de auditoría

La guía exige al menos dos técnicas (Vacca, 2026); se usaron siete, todas automatizadas con scripts en `auditoria/scripts/` que guardan su salida con fecha y commit para poder repetir la prueba después de cada corrección (Tabla 3). Las pruebas dinámicas se hicieron con el APK de release instalado en un emulador Android 15; para las que necesitaban sesión, un integrante del grupo ingresó directamente en el emulador una cuenta de prueba creada para la auditoría. Las reglas se probaron en el emulador local de Firebase con un proyecto de demostración.

Tabla 3

Técnicas, herramientas y riesgos cubiertos

Técnica	Herramienta	Riesgos	Alcance
Secretos en código e historial	gitleaks 8.30.1 (Rice, 2025)	M1, M10	81 commits, todas las ramas y los archivos actuales
Dependencias	`flutter pub outdated`, `npm audit`	M2	Paquetes de Flutter y de Cloud Functions
Manifiesto y permisos	Script propio y `apkanalyzer`	M5, M6, M8	Manifiesto fuente y del APK, `Info.plist`
Análisis del APK de release	`apkanalyzer`, `apksigner`	M7, M8	Firma, depurable, símbolos en `libapp.so`, R8
Análisis estático automatizado	MobSF v4.6.5 en Docker (MobSF, 2025)	M1, M5, M7, M8, M9, M10	APK de release antes y después, por su API REST
Revisión de código	Lectura de reglas y 14 Cloud Functions; emulador de Firebase	M3, M4, M9	Autorización, validación, almacenamiento
Ejecución en emulador	Logcat y mitmproxy 12.3.1 (mitmproxy Project, 2025)	M5, M6, M9	Arranque, inicio de sesión con y sin interceptor, pantallas con el rostro

Nota. Las evidencias de la sesión no incluyen la contraseña de la cuenta de prueba, que se eliminará al terminar la auditoría.

Resultados

Revisión del OWASP Mobile Top 10

La Tabla 4 resume el estado de cada riesgo en la versión auditada. Tres riesgos cumplen y siete no cumplen; el detalle de cada uno se presenta a continuación.

Tabla 4

Estado de los riesgos del OWASP Mobile Top 10 (2024) en la versión auditada

ID	Riesgo	Estado	Hallazgos
M1	Improper Credential Usage	No cumple	H-01
M2	Inadequate Supply Chain Security	No cumple	H-05
M3	Insecure Authentication/Authorization	No cumple	H-03, H-06
M4	Insufficient Input/Output Validation	Cumple	—
M5	Insecure Communication	Cumple	—
M6	Inadequate Privacy Controls	No cumple	H-07, H-08
M7	Insufficient Binary Protections	No cumple	H-04
M8	Security Misconfiguration	No cumple	H-01, H-07
M9	Insecure Data Storage	No cumple	H-02, H-09
M10	Insufficient Cryptography	Cumple	—

Nota. Riesgos según OWASP Foundation (2024a).

M1. Improper Credential Usage: no cumple

gitleaks marcó como `gcp-api-key` cuatro llaves de cliente de Firebase en el historial, en `lib/firebase_options.dart` y `android/app/google-services.json`. Google las trata como identificadores públicos cuyo acceso controlan las reglas (Google, s. f.-a), por lo que se aceptaron como falso positivo con la condición de restringirlas. La llave de Gemini está en Secret Manager y no aparece en el código. Sí hay una credencial expuesta: `debug.keystore`, con su contraseña escrita en Gradle, firmaba también el release (H-01).

```
gitleaks git --log-opts=--all   →  81 commits scanned · leaks found: 6
  4  gcp-api-key  lib/firebase_options.dart
  2  gcp-api-key  android/app/google-services.json
```

M2. Inadequate Supply Chain Security: no cumple

`npm audit` en `functions/` reportó diez vulnerabilidades transitivas, una de ellas alta en `body-parser` (H-05). En Flutter había versiones mayores pendientes de Firebase y ML Kit sin avisos de seguridad publicados.

M3. Insecure Authentication/Authorization: no cumple

Las once funciones invocables exigen sesión y las de administración verifican el claim `admin` en el servidor. Las reglas limitan cada colección a su dueña e impiden asignarse un rol desde el cliente. Sin embargo, una marca desactivada conservaba permisos de escritura y retirar el rol no revocaba la sesión (H-03). Tampoco había App Check (H-06), el mecanismo de Firebase para aceptar solo peticiones de la app legítima (Google, s. f.-b).

M4. Insufficient Input/Output Validation: cumple

Cada función valida tipos, largos y listas blancas: mensajes de chat de hasta 500 caracteres, límite de mensajes por minuto, identificadores sin barras y rutas de archivos temporales con un patrón estricto. Las reglas validan tipos y enumeraciones, y la app no declara deep links.

M5. Insecure Communication: cumple

La app no habilita tráfico en claro; con `targetSdk` 36 está bloqueado por defecto y todo el tráfico va por HTTPS a servicios de Google. En la versión auditada no había certificate pinning; se agregó después como mejora opcional (ver Correcciones). La prueba dinámica lo confirma: con mitmproxy interceptando el tráfico, el inicio de sesión falla con `CertPathValidatorException` y el proxy registra 18 conexiones rechazadas porque la app no confía en su certificado.

```
mitmdump:  Client TLS handshake failed. The client does not trust the proxy's certificate
           for firebaseinstallations.googleapis.com   (18 conexiones rechazadas)
logcat:    RecaptchaAction(action=signInWithPassword) ... CertPathValidatorException:
           Trust anchor for certification path not found
```

M6. Inadequate Privacy Controls: no cumple

La colorimetría pide consentimiento y las selfies se borran después del análisis. Sin embargo, la app declaraba permisos de galería, micrófono y biometría que no usa (H-07) y cualquier usuaria podía leer la foto de perfil de otra (H-08).

M7. Insufficient Binary Protections: no cumple

El código Dart se compilaba sin ofuscar (H-04). R8 sí estaba activo en la parte Java del APK.

```
APK release (28fc2f7) — lib/arm64-v8a/libapp.so
  rutas "package:mackeupai/...dart" legibles:  209
  nombres: CredencialRepository · CapturasArRepository · asignarAdminMarca
classes.dex: 989 clases renombradas por R8 · apkanalyzer manifest debuggable → false
```

M8. Security Misconfiguration: no cumple

El release no es depurable y el único componente exportado propio es la actividad principal. Sin embargo, el release se firmaba con la llave de depuración pública (H-01) y pedía permisos de más (H-07).

M9. Insecure Data Storage: no cumple

`flutter_secure_storage` solo guarda el correo recordado y nunca contraseñas ni tokens. Sin embargo, las capturas AR del rostro se guardaban sin cifrar y entraban en el respaldo de Android (H-02), y 16 llamadas a `debugPrint` escribían en Logcat en release (H-09).

M10. Insufficient Cryptography: cumple

No se usan MD5, SHA-1, DES ni ECB, ni hay llaves embebidas. El único hash propio es SHA-256 para seudonimizar estadísticas en el servidor.

Hallazgos

La Tabla 5 registra los nueve hallazgos con el formato que pide la guía (Vacca, 2026), ordenados por severidad.

Tabla 5

Registro de hallazgos de la auditoría

ID	OWASP	Descripción	Evidencia	Sev.	Corrección propuesta
H-01	M1, M8	El release se firma con `debug.keystore`, publicado en el repositorio con su contraseña.	`build.gradle.kts` L31-44; `apksigner`: `CN=Android Debug`	Alta	Llave de release fuera del repositorio.
H-02	M9	Capturas AR del rostro sin cifrar e incluidas en el respaldo de Android.	`allowBackup` sin definir; `dumpsys`: `ALLOW_BACKUP`	Media	Desactivar el respaldo y excluir los datos.
H-03	M3	Marca desactivada o ex-admin siguen escribiendo en el catálogo.	`firestore.rules` L131-139; `marcas.ts` L367	Media	Exigir marca activa y correo en `admins`; revocar sesión.
H-04	M7	Código Dart sin ofuscar en release.	209 rutas legibles en `libapp.so`	Media	Compilar con `--obfuscate`.
H-05	M2	Diez vulnerabilidades en dependencias de Functions.	`npm audit`: 1 alta, 8 moderadas, 1 baja	Media	`npm audit fix` y actualizar `firebase-admin`.
H-06	M3, M1	Backend sin App Check.	Sin `app_check` en el código	Media	App Check con Play Integrity.
H-07	M6, M8	Permisos que la app no usa (galería, micrófono, biometría).	Manifiesto del APK: 14 permisos	Baja	Quitarlos con `tools:node="remove"`.
H-08	M6	Fotos de perfil legibles por cualquier usuaria.	`storage.rules` L18-19	Baja	Lectura solo de la dueña o la admin.
H-09	M9	16 `debugPrint` activos en release.	Revisión de código	Baja	Silenciar `debugPrint` en release.

Correcciones

Se corrigieron ocho de los nueve hallazgos, empezando por el de mayor severidad, cada uno en su propio commit con su identificador en el mensaje (Tabla 6). Antes de cada commit se ejecutaron `flutter analyze`, `flutter test` (326 pruebas) y `npm test` (51 pruebas). Las correcciones siguen las guías de Android sobre respaldo automático, selector de fotos y firma de apps (Android Developers, s. f.-a, s. f.-c, s. f.-d) y la de Flutter sobre ofuscación (Flutter, s. f.).

Tabla 6

Correcciones aplicadas y mejora obligatoria que cubren

ID	Mejora obligatoria	Cambio	Commit
H-01	Ningún secreto en el repositorio	Release firmado con una llave propia (`key.properties` y `.jks` ignorados por Git); Gradle se detiene si falta.	`e85e1aa`
H-03	Permisos validados en el servidor	Reglas exigen marca activa y correo en `admins`; se revocan los refresh tokens al cambiar el rol.	`bca8a32`
H-02	Datos sensibles protegidos	`allowBackup="false"` y reglas de extracción que excluyen todo.	`868452b`
H-04	Release ofuscado y con R8	Build con `--obfuscate --split-debug-info`; símbolos fuera del repositorio.	`4d1386e`
H-09	Release sin logs sensibles	`debugPrint` vacío en release (`lib/main.dart`).	`f88462c`
H-05	Dependencias sin vulnerabilidades	`npm audit fix` y `firebase-admin` 14.5.1.	`0e8c8f5`
H-07	Mínimo privilegio	Se quitan seis permisos que la app no usa.	`2738357`
H-08	Mínimo privilegio	Foto de perfil legible solo por su dueña o la admin.	`32401ed`

Nota. H-06 no se implementó porque App Check exige registrar la app en Play Integrity desde la consola de Firebase. Las reglas y funciones corregidas deben publicarse con `firebase deploy`.

Mejoras preventivas adicionales

Además de los hallazgos, se implementaron dos controles de la tabla de controles por plataforma de la guía (Vacca, 2026), uno de ellos la mejora opcional de certificate pinning (Tabla 7). El pinning sigue la configuración de seguridad de red de Android (Android Developers, s. f.-b): se fijan las raíces de Google Trust Services, que vencen en 2036, y no los certificados intermedios, que Google rota con frecuencia; hay pines de respaldo, y los pines vencen el 30 de junio de 2027 para que, si Google cambia de autoridad, la app vuelva a la validación normal en vez de quedarse sin conexión. El script `verificar_pines.ps1` comprueba las cadenas reales antes de renovarlos.

Tabla 7

Mejoras preventivas implementadas

Mejora	OWASP	Cambio	Commit
Certificate pinning con plan de rotación	M5	`network_security_config.xml`: sin tráfico en claro; `*.googleapis.com` y `*.cloudfunctions.net` fijados a GTS Root R1–R4 y GlobalSign Root CA; vencimiento 2027-06-30.	`793f84a`
`FLAG_SECURE` en pantallas con el rostro	M6, M9	La selfie, el simulador AR y las capturas AR bloquean capturas y grabaciones de pantalla y no se ven en "apps recientes".	`3a0f001`

Verificación

Se repitió la misma prueba de cada hallazgo sobre la versión auditada y sobre la rama corregida (Tabla 8). Un hallazgo solo se consideró cerrado si la prueba repetida ya no lo reproducía.

Tabla 8

Resultado de las pruebas antes y después de cada corrección

ID	Prueba	Antes	Después	Cerrado
H-01	`apksigner verify`	`CN=Android Debug` (llave del repositorio)	`CN=MackeUp AI, O=UNICESAR` (llave propia)	Sí
H-02	`dumpsys package`	Bandera `ALLOW_BACKUP`	Sin `ALLOW_BACKUP`	Sí
H-03	Emulador de reglas	Ex-admin y marca desactivada escriben (200)	Ambas reciben 403; la admin vigente, 200	Sí
H-04	Rutas en `libapp.so`	209 rutas y nombres de clases	0 rutas; sin nombres de clases	Sí
H-05	`npm audit`	10 (1 alta)	7 moderadas, 0 altas	Parcial
H-07	Permisos del APK	14, con micrófono y almacenamiento	8, sin micrófono, almacenamiento ni biometría	Sí
H-08	Emulador de reglas	Otra usuaria lee la foto (200)	Otra usuaria, 403; la dueña, 200	Sí
H-09	Código y Logcat del release	16 `debugPrint` activos	Ninguna línea de la app en Logcat	Sí
Pinning	`aapt2 dump xmltree` del APK y `verificar_pines.ps1`	Sin `network_security_config`	5 pines en el APK; los 7 dominios encadenan a un pin; sin interceptor el inicio de sesión funciona; con mitmproxy falla (`CertPathValidatorException`)	Sí
`FLAG_SECURE`	Pruebas automáticas y `screencap` en el emulador	Capturas permitidas en todas las pantallas	6 pruebas nuevas; la captura de la pantalla de la selfie sale en negro, solo se ve el diálogo del sistema	Sí

Nota. La prueba de reglas usa el emulador local de Firebase con el proyecto `demo-auditoria`, sin acceso al proyecto real.

```
probar_reglas.ps1 -Commit 28fc2f7  →  Resultado: 4 caso(s) no cumplen
probar_reglas.ps1  (corregido)     →  Resultado: todas las reglas se comportan como se espera
npm audit   antes: 10 vulnerabilities (1 low, 8 moderate, 1 high)   después: 7 moderate
```

Como verificación independiente, los dos APK de release se analizaron con MobSF (Tabla 9). La herramienta confirma de forma automática tres de las correcciones: desaparecen la alerta alta de firma con certificado de depuración (H-01) y el aviso de respaldo de datos (H-02), y los permisos peligrosos bajan de seis a dos (H-07).

Tabla 9

Resultado del análisis con MobSF antes y después de las correcciones

Indicador	Antes (`28fc2f7`)	Después (`4d1386e`)
Puntaje de seguridad	48/100	52/100
Alertas altas	2: firma con certificado de depuración; `minSdk` 24	1: `minSdk` 24
Avisos	15, incluido `allowBackup` sin definir	14, sin el aviso de `allowBackup`
Permisos peligrosos	6: cámara, notificaciones, micrófono, imágenes, lectura y escritura de almacenamiento	2: cámara y notificaciones
Secretos detectados	Llave de cliente de Firebase y tablas hexadecimales de librerías	Igual (falsos positivos)

Nota. MobSF analiza todo el código del APK, incluidas las librerías de Google. Los avisos de MD5, SHA-1, generador aleatorio inseguro, SQLite y archivos temporales provienen de esas librerías: el código de la app no usa esos algoritmos (ver M10). Los componentes exportados que reporta son de Firebase y están protegidos por permisos del sistema. Reportes completos en `auditoria/evidencia/mobsf_*`.

Discusión

La base de seguridad de MackeUp AI es sólida: la autorización y la validación de entradas se hacen en el servidor, los secretos del backend están en Secret Manager y el dispositivo no guarda contraseñas ni tokens. Los problemas encontrados fueron sobre todo de configuración del build y de permisos, que suelen pasar desapercibidos porque la app funciona igual con ellos.

El hallazgo H-05 quedó cerrado de forma parcial. Las siete vulnerabilidades moderadas restantes están anidadas en los SDK de Google (`gaxios`, `uuid`, `rimraf`, `glob` y `minimatch`) y solo podrían eliminarse forzando versiones mayores, con riesgo de romper las funciones en producción. Ninguna tiene una ruta de ataque en este backend, por lo que se aceptaron como riesgo residual que se revisará con cada actualización de `firebase-admin`.

MobSF señaló además una alerta que no estaba entre los hallazgos: la app admite Android 7.0 (`minSdk` 24), una versión que ya no recibe parches de seguridad. Subir la versión mínima reduce ese riesgo pero deja fuera teléfonos antiguos, por lo que se deja como decisión del equipo en las recomendaciones.

Durante la prueba con sesión iniciada apareció una observación más: el SDK de Firebase Authentication escribe en Logcat el correo de la usuaria al iniciar sesión (`Logging in as …`). No viene del código de la app, que ya no registra nada en release (H-09), sino de la librería de Google, por lo que queda como riesgo residual de M9.

Como limitación, la prueba con mitmproxy se hizo con el APK de release, que no confía en certificados instalados por el usuario: demuestra que la app rechaza una interceptación, pero no aísla el efecto del pinning frente a la validación normal. Para aislarlo, la configuración incluye una excepción solo para builds de depuración que acepta certificados de usuario sin desactivar los pines, de modo que la misma prueba sobre un build de depuración con el certificado de mitmproxy instalado debe fallar únicamente por el pinning.

Conclusiones

Se cumplieron los cuatro objetivos específicos. El modelo de amenazas identificó cuatro amenazas propias de la aplicación, la revisión del OWASP Mobile Top 10 se hizo con siete técnicas y produjo nueve hallazgos con evidencia, se corrigieron ocho de ellos (más del mínimo de cinco que pide la guía) y cada corrección quedó en su propio commit y verificada con la misma prueba antes y después. Se añadieron además certificate pinning con plan de rotación, la mejora opcional de la guía, y la protección `FLAG_SECURE` de las pantallas que muestran el rostro.

El hallazgo más grave, la firma del release con una llave publicada en el repositorio público, muestra que un secreto no siempre es una contraseña en el código: también lo es una llave de firma, y su exposición permite suplantar la aplicación. Su corrección, junto con la del respaldo de datos y la de los permisos de marca, reduce de forma concreta las amenazas A-01, A-02 y A-03 del modelo.

La auditoría confirmó además que validar en el servidor es el control que más protege la aplicación: las reglas y las Cloud Functions ya impedían los abusos más evidentes, y los fallos encontrados en ellas se pudieron demostrar y cerrar con pruebas automáticas en el emulador. Finalmente, no toda alerta de una herramienta es explotable: las siete vulnerabilidades residuales de las dependencias se analizaron y se aceptaron de forma justificada en lugar de forzar cambios que podían romper la aplicación en producción.

Recomendaciones

1. Publicar las reglas y funciones corregidas con `firebase deploy --only firestore:rules,storage,functions`, porque hasta entonces H-03 y H-08 solo están corregidos en el repositorio.
2. Crear la llave de publicación definitiva con `crear_llave_release.ps1`, guardarla en un lugar seguro y registrar sus huellas SHA-1 y SHA-256 en Firebase.
3. Integrar Firebase App Check con Play Integrity y restringir las llaves de API por paquete y huella (H-06).
4. Eliminar la cuenta de prueba usada en la verificación y repetir la prueba de mitmproxy sobre un build de depuración con el certificado instalado para aislar el efecto del pinning.
5. Evaluar subir el `minSdk` de 24 (Android 7.0) a una versión que aún reciba parches de seguridad, según la alerta de MobSF, considerando cuántas usuarias tienen teléfonos antiguos.
6. Revisar `npm audit` y `flutter pub outdated` en cada actualización de dependencias, y volver a correr los scripts de `auditoria/scripts/` antes de cada entrega.

Referencias

Android Developers. (s. f.-a). *Back up user data with Auto Backup*. Google. https://developer.android.com/identity/data/autobackup

Android Developers. (s. f.-b). *Network security configuration*. Google. https://developer.android.com/privacy-and-security/security-config

Android Developers. (s. f.-c). *Photo picker*. Google. https://developer.android.com/training/data-storage/shared/photopicker

Android Developers. (s. f.-d). *Sign your app*. Google. https://developer.android.com/studio/publish/app-signing

Flutter. (s. f.). *Obfuscate Dart code*. Google. https://docs.flutter.dev/deployment/obfuscate

Google. (s. f.-a). *Learn about using and managing API keys for Firebase*. Firebase. https://firebase.google.com/docs/projects/api-keys

Google. (s. f.-b). *Firebase App Check*. Firebase. https://firebase.google.com/docs/app-check

Holloway-George, E. (2024, febrero). *What's the buzz about the 2024 OWASP Mobile Top 10 changes?* ProAndroidDev. https://proandroiddev.com/whats-the-buzz-about-the-2024-owasp-mobile-top-10-changes-83c93f765bd3

mitmproxy Project. (2025). *mitmproxy* (Versión 12.3.1) [Software]. https://mitmproxy.org

MobSF. (2025). *Mobile Security Framework (MobSF)* (Versión 4.6.5) [Software]. GitHub. https://github.com/MobSF/Mobile-Security-Framework-MobSF

OWASP Foundation. (2024a). *OWASP Mobile Top 10*. https://owasp.org/www-project-mobile-top-10/

OWASP Foundation. (2024b). *OWASP Mobile Application Security Verification Standard (MASVS)* (Versión 2.1.0). https://mas.owasp.org/MASVS/

OWASP Foundation. (2026). *OWASP Mobile Application Security Testing Guide (MASTG)* (Versión 2.0.0). https://mas.owasp.org/MASTG/

Rice, Z. (2025). *Gitleaks* (Versión 8.30.1) [Software]. GitHub. https://github.com/gitleaks/gitleaks

Vacca, A. (2026). *Tarea: Seguridad en aplicaciones móviles* [Guía de actividad, Programación Móvil]. Universidad Popular del Cesar.